

Modo IA Tudo Imagens Vídeos Shopping Mais ▾



-
2. **Os Subagentes (Operários):** O agente principal abre um subagente e diz: *"Pegue a decisão X que está no arquivo Y, implemente o teste unitário para a função Z e feche a sessão"*.
3. **O Fechamento:** Assim que o subagente termina a tarefa, **a sessão dele é destruída**. Os tokens daquela discussão somem, mas o código gerado e o progresso ficam salvos no seu repositório.

Dessa forma, o seu "contexto principal" não fica poluído com milhares de linhas de código de refatoração ou logs de erro de testes que falharam. Tudo isso acontece isolado na caixinha do subagente.

Como evitar que sua conta de tokens exploda

Para garantir que a estratégia reduza seus custos em vez de aumentá-los, adote estas práticas:

- **Use a arquitetura como âncora:** Em vez de confiar na memória da conversa, faça o Claude Code documentar as decisões em arquivos de texto no próprio projeto (ex: `docs/architecture_decisions.md`). Sempre que reiniciar ou chamar um subagente, a referência estará no arquivo, não no histórico de chat.
- **Microtarefas:** Nunca dê comandos abertos como *"melhore o sistema"* para um subagente. Use comandos diretos: *"Refatore a função de login usando o padrão definido em docs/patterns.md"*.
- **Monitore o cache de contexto:** O Claude da Anthropic possui um recurso de **Context Caching**. Se o seu agente principal reutilizar as instruções de sistema e arquivos de documentação fixos, esses tokens entram em preço de cache (que é muito mais barato). O subagente entra, faz o trabalho rápido e sai antes de acumular custos de histórico.

Para eu te ajudar a configurar isso melhor no seu dia a dia, me conta:



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



Tá, faz sentido... Mas como exatamente os subagentes ajudam nisso? Porque se eu só dividir em várias sessões... ▾

A confusão faz todo sentido, porque a matemática parece não fechar.

Se cada subagente tem seu contexto, como isso economiza?

O segredo está em **como** os tokens são cobrados nos LLMs: **você paga pelo histórico inteiro a cada nova mensagem que envia.**

Para entender a economia e como os contextos não ficam soltos, pense no subagente não como um chat paralelo que você abre no navegador, mas como uma **função de código isolada** que o agente principal gerencia automaticamente.

1. Por que isso economiza tokens (A Matemática do Histórico)

Imagine que você está desenvolvendo uma funcionalidade ao longo do dia. O seu prompt de sistema + arquivos base somam **10.000 tokens** fixos.

Cenário A: Sessão Única (O que você faz hoje)

A cada mensagem, o histórico cresce. Na mensagem número 30, o histórico acumulou mais **30.000 tokens** de conversas passadas, erros, tentativas e códigos antigos.

- **Mensagem 31:** Você envia uma pergunta simples ("Corrija o ponto e vírgula na linha 10").
- **O custo:** Você paga os 10.000 (base) + 30.000 (histórico) + o seu



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



- O subagente nasce apenas com o contexto necessário: os 10.000 tokens base + o arquivo da linha 10.
- Ele faz a correção em **1 ou 2 mensagens** (gasta uns 11.000 tokens).
- O subagente faz o commit/alteração no arquivo e **é deletado**.
- **O resultado:** Aqueles 30.000 tokens de "conversa fiada" e erros de compilação do dia **nunca existiram** para o Agente Principal. O histórico macro continua limpo e barato.

2. Por que os contextos não ficam soltos?

Os contextos não ficam soltos porque **o ponto de união e sincronização não é a memória do Claude, é o seu sistema de arquivos (o código do projeto)**.

No Claude Code, os subagentes são acionados via ferramentas de terminal (`tools`). O fluxo funciona assim de forma automatizada:

```
[Seu Terminal: Agente Principal]
| (Guarda as regras de arquitetura e o plano macro)
|
|→ [Cria Subagente 1] → Lê o arquivo A → Escreve o código
|
|→ [Cria Subagente 2] → Lê o arquivo A (já atualizado!) -
```

1. **A Linha de Base é o Código:** O Subagente 1 altera um arquivo e morre. Quando o Subagente 2 é criado logo em seguida para fazer outra tarefa, ele não precisa saber *o que o Subagente 1 conversou com você*. Ele só precisa **ler o arquivo modificado** no disco. O código atualizado é a única



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



Em vez de pagar o Claude para carregar uma mochila de 50.000 tokens de histórico nas costas o dia todo (o que faz ele "amolecer" e esquecer as regras), os subagentes permitem que ele trabalhe sempre com uma mochila leve de 5.000 ou 10.000 tokens, executando tarefas rápidas, salvando no arquivo e limpando a memória.

Para podermos aplicar isso na sua rotina com o Claude Code:

- Qual é o **tamanho aproximado do projeto** (linhas de código ou número de arquivos)?
- Você costuma usar os comandos dele direto no **terminal** ou deixa ele rodando no modo **interativo** por muito tempo?



Entendi, então o subagente é tipo uma função isolada que faz o trabalho sozinho e devolve só o resultado final, se... ▾

Você pegou exatamente o espírito da coisa! É exatamente isso: uma função isolada que executa a tarefa, altera o arquivo e desaparece.

Sobre a sua dúvida: **sim, isso resolve de verdade o problema da degradação** e não apenas o empurra para outra janela. Há duas razões fundamentais para isso:

1. A janela do subagente é curta demais para degradar

O "amolecimento" do Claude não acontece por causa da complexidade do código em si, mas sim pelo **tamanho do histórico de conversas (chat)**.

A janela do subagente é curta demais para degradar



Quando essa microtarefa falha ou termina, aquela janela é descartada. O Agente Principal pode simplesmente abrir um *novo* subagente do zero. Esse novo subagente virá com a "mente" 100% limpa, sem o estresse ou os vícios do subagente anterior, lendo apenas o código atual e as instruções limpas do prompt de sistema.

Situação	No modelo tradicional (Sessão Única)	Com Subagentes (Arquitetura Claude Code)
Erros de sintaxe/linter	Poluem o chat principal. O Claude começa a ignorar as regras de estilo após 5 tentativas de correção.	O subagente lida com os erros de linter isoladamente. O chat principal só vê o código final corrigido.
Esquecer o padrão de código	Conforme o chat cresce, ele esquece de usar <code>camelCase</code> ou a estrutura que você pediu lá na primeira mensagem.	Toda vez que um subagente abre, ele recebe o arquivo de regras limpo como prioridade máxima (sem histórico para distraí-lo).
Excesso de arquivos abertos	O Claude começa a misturar o contexto de tabelas do banco com regras de estilização CSS.	O subagente só recebe acesso aos arquivos estritamente necessários para aquela microtarefa.

Em resumo: em vez de você ter uma única sessão gigante que vai ficando "burra" e cara com o passar das horas, você tem um **Agente Principal que nunca degrada** (porque o histórico dele quase não cresce, já que o trabalho

Modo IA Tudo Imagens Vídeos Shopping Mais ▾



Beleza, então o subagente faz a tarefa, some e mantém o contexto principal limpo. Mas e o custo? Porque se eu ... ▾

A preocupação com o custo é o ponto mais crítico e você está coberto de razão em questionar: **se não houver controle, o custo pode sim aumentar**.

No entanto, existe um "ponto de virada" matemático onde os subagentes passam a ser **mais baratos** do que a sessão única.

Para usar essa estratégia de forma inteligente, você precisa entender onde fica esse limite e como jogar com as regras de cobrança da Anthropic.



O Ponto de Virada: Onde a matemática muda a seu favor

A cobrança dos LLMs funciona de forma **acumulativa**. Você paga por cada token enviado a cada nova mensagem. Veja a diferença de progressão de custos:

- **Na Sessão Única (Custo Exponencial):** Se o seu contexto base é de 10.000 tokens e você troca 30 mensagens de 1.000 tokens cada, a mensagem número 31 vai te custar **41.000 tokens de entrada**. O custo cresce como uma bola de neve, porque você paga pelo passado repetidamente.
- **Com Subagentes (Custo Linear):** O Agente Principal consome quase nada (ele só gerencia o plano). Cada subagente abre com as mesmas



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



Onde está o perigo? (O que faz a conta explodir)

O subagente se torna um vilão financeiro se você cometer o erro do **repassse cego de contexto**.

Se o Agente Principal for "preguiçoso" e, ao criar o subagente, passar o projeto inteiro (ex: 50 arquivos, logs imensos e arquivos de mídia desnecessários), você estará pagando o custo de inicialização de uma janela gigante toda vez. O segredo da economia é a **dieta de contexto**.

Regras de ouro para usar subagentes sem falir

Para garantir que você fique do lado econômico da linha, siga estas três regras práticas no Claude Code:

- Aproveite o Context Caching (Preço Reduzido):** A Anthropic oferece desconto (de até 90%) para tokens que ficam guardados no cache. Deixe seus arquivos de regras estruturais (`.claudecode/instructions`, documentação de arquitetura) fixos. O Claude Code gerencia isso automaticamente: o subagente vai ler esses arquivos direto do cache, custando uma fração do preço normal.
- Restrinja o escopo por comando:** Nunca diga *"Subagente, ajuste o módulo de usuários"*. Diga: *"Subagente, altere apenas o arquivo `src/users/controller.ts` para adicionar a validação do campo X"*. Menos arquivos lidos = menos tokens de entrada faturados.
- Limite o número de iterações (`max_iterations`):** Ao rodar comandos autônomos no Claude Code, você pode definir limites de passos ou usar o modo interativo onde você autoriza cada loop do subagente. Não deixe o subagente rodando em loops infinitos tentando corrigir um erro bizarro.



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



- Quanto você costuma gastar de **créditos/tokens** em uma sessão longa típica de trabalho hoje?
- Seus projetos costumam ter arquivos muito grandes (mais de 1.000 linhas por arquivo) ou são divididos em pequenos componentes?



Hmm, entendi. Então, se eu decidir usar subagentes, é melhor focar em tarefas maiores e mais bagunçadas, ... ▾

Você pegou **exatamente** a lógica econômica da coisa!

Para tarefas rápidas e diretas (onde ele acerta de primeira), o agente principal resolve direto. O subagente brilha justamente nas **tarefas maiores, iterativas e "bagunçadas"** (que envolvem rodar testes, debugar erros de linter ou refatorar lógica pesada) [Claude Code]. É aí que o custo de inicialização dele compensa de longe a economia de manter o contexto principal limpo e inteligente.

E sobre a sua segunda pergunta: **sim, com certeza!** Configurar subagentes para papéis altamente especializados (como um revisor de segurança, um especialista em performance ou um gerador de testes unitários) é uma das estratégias mais avançadas e eficazes do Claude Code [Claude Code].

Como criar subagentes especializados na prática

No Claude Code, você não precisa programar um sistema complexo para fazer



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



"Você é um subagente especialista em segurança AppSec. Sua única função é analisar o arquivo fornecido em busca de vulnerabilidades (SQL Injection, XSS, falhas de autenticação e vazamento de segredos). Não altere a lógica de negócios, foque apenas em mitigar riscos. Se encontrar algo, reescreva a função aplicando práticas seguras."

- **Como você usa:** No chat principal, você digita:
`@security-agent revise as alterações que fiz no arquivo src/auth/login.ts before do commit.` [Claude Code]
- **O benefício:** O subagente nasce com o "mindset" focado apenas em segurança, sem se distrair com o resto do código.

 Exemplo 2: O Criador de Testes (`.claudecode/test-agent.md`)

"Você é um engenheiro de QA automatizado. Você deve ler o arquivo de código-fonte fornecido e gerar um arquivo de teste equivalente usando Jest/Vitest. Garanta 100% de cobertura nas ramificações (edge cases). Rode o comando `npm test` para garantir que passam."

- **Como você usa:** Quando terminar uma feature no agente principal, você manda:
`@test-agent crie os testes unitários para a nova função de pagamento em src/services/payment.ts.` [Claude Code]
- **O benefício:** Toda a "bagunça" de rodar os testes no terminal, ver o teste falhar, corrigir a sintaxe e rodar de novo acontece **dentro do subagente**. O seu contexto principal nunca vê esses erros e continua intacto.



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



2. **Adesão Estrita aos Padrões:** Como ele não carrega o histórico de você mudando de ideia sobre a regra de negócio nas últimas 3 horas, ele aplica os padrões de segurança ou de testes de forma cirúrgica e fria.

O Próximo Passo Inteligente

Para começar a testar isso sem fricção, o ideal é criar o seu primeiro subagente especializado para a tarefa que você considera a mais chata ou propensa a erros no seu dia a dia.

Se quiser avançar nisso, me conte:

- Qual seria o **papel ideal** para o seu primeiro subagente especializado? (Ex: **Revisor de Segurança**, **Especialista em Performance**, **Escritor de Testes**, ou **Gerador de Documentação**?)
- Qual é a **stack tecnológica** (Node.js, Python, Go, etc.) e a **ferramenta de testes/linter** que você usa no projeto?

Com isso, posso te ajudar a escrever o prompt de sistema perfeito para esse subagente!



Boa, então dá pra criar subagentes bem focados e até compartilhar com o time pra otimizar certas tarefas. Iss... ▾

Compartilhar esses subagentes especializados com o time é uma das melhores jogadas!

Como eles ficam salvos como arquivos de texto (Markdown) dentro do próprio repositório (ex: na pasta `claudecode/`), qualquer desenvolvedor da equipe



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



... e depois, quando a sessão estiver fechada, eu posso te ajudar a fechar a sessão ou isolar a tarefa em um subagente focado [Claude Code].

Para quando você for colocar a mão na massa, se quiser, posso te ajudar com o primeiro passo:

- **Estruturar o esqueleto do prompt** para o seu primeiro subagente especializado (como o de testes ou segurança).
- **Definir a melhor estratégia de Context Caching** para as regras do seu projeto específico para garantir o menor custo possível.

Como você prefere começar a estruturar esse fluxo de testes?



Vender com honestidade o trade-off: Subagentes não deixam o Claude "mais inteligente" — eles mantêm o ...



Perfeito!

Você resumiu com **total precisão e honestidade** o verdadeiro funcionamento dessa arquitetura.

Não existe mágica de inteligência: o modelo de IA subjacente é rigorosamente o mesmo. A melhora na qualidade acontece puramente porque **um contexto limpo e sem ruído permite que o modelo performe no seu nível máximo**, sem o "peso morto" de um histórico longo e confuso.

E o trade-off financeiro é exatamente esse: **cada subagente novo paga o pedágio do contexto inicial**. Se você fragmentar seu fluxo de trabalho chamando subagentes para cada linha de código ou tarefa boba, a sua conta



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



Se você quiser, quando for rodar os seus testes práticos, posso te ajudar a **desenhar um fluxo visual** ou **escrever o esqueleto de instruções** para o seu primeiro subagente de tarefa pesada. Qual cenário do seu dia a dia você acha que se encaixa melhor nessa descrição de "tarefa grande e ruidosa"?



Gostei dessa regra de ouro, bem pragmática! No meu dia a dia, acho que o melhor cenário pra começar seria com... ▾

Começar por revisões profundas de segurança (AppSec) é a escolha perfeita.

É um cenário clássico onde o modelo precisa de foco cirúrgico e que costuma gerar longos debates ("E se o input vier no formato X?", "Como mitigar o bypass do linter?"), o que destruiria o contexto da sua sessão principal se fosse feito lá.

Para você colocar isso em prática imediatamente, eu estruturei as regras, o prompt de sistema ideal e o fluxo de comando econômico em um arquivo Markdown pronto para o seu repositório.

Aqui está o seu arquivo:

Security-Agent

 MD

Ele contém o **System Prompt** focado em segurança, as instruções de como









Como usar esse arquivo no seu dia a dia:

- 1. Salve o conteúdo do arquivo gerado em uma pasta do seu projeto, por exemplo, em `.claudecode/security-agent.md`.
- 2. Quando criar um endpoint crítico (ex: um controlador de login ou de transações), não peça para o Claude principal revisar. Chame o subagente direto pelo terminal apontando para este arquivo [Claude Code]:

```
bash
claude "leia as instruções em .claudecode/security-agent.md"
```

Use o código com cuidado.

- 3. O subagente vai abrir, aplicar a mentalidade de um hacker ético,

+

🎤

Modo IA Tudo Imagens Vídeos Shopping Mais ▾



biblioteca de autenticação que vocês usam? Posso incluir regras específicas para o subagente caçar falhas comuns deles!



Ah, isso ficou bem prático e direto ao ponto! A gente usa PostgreSQL como banco de dados e a biblioteca de ... ▾

Ajustei o arquivo para torná-lo um especialista cirúrgico na sua stack.

Agora ele inclui checagens específicas para blindar o **PostgreSQL** contra SQL Injection (especialmente em queries brutas) e garantir que o ciclo de vida do **Passport.js** esteja 100% seguro (sessões, JWTs e serialização).

Aqui está o seu arquivo atualizado e pronto para uso:

 O Arquivo do Subagente (`.claudecode/security-agent.md`)

markdown

```
# Role: Subagente Especialista em AppSec (Node.js + PostgreSQL +  
  
Você é um subagente focado estritamente em segurança de código (  
Sua missão é analisar o arquivo fornecido pelo usuário e identif  
  
## 🎯 Escopo de Atuação (Foco Total)  
1. Analise APENAS o arquivo indicado. Não tente ler ou alterar o  
2. Não altere regras de negócio, formatação de código ou estilos  
3. Se encontrar vulnerabilidades, corrija-as diretamente no arqu
```



Modo IA Tudo Imagens Vídeos Shopping Mais ▾



3. Vulnerabilidades Gerais OWASP Top 10

- Falhas de controle de acesso (Broken Object Level Authorizati
- Entrada de dados não sanitizada (XSS).
- Vazamento de segredos/chaves de API no código.

🛠️ Output Esperado

Se o arquivo estiver seguro, responda apenas: "✅ Arquivo analisado"
Se houver problemas, aplique a correção direto no código do arquivo

Use o código com cuidado.



Como rodar o seu primeiro teste prático

Abra o terminal na raiz do seu projeto e execute o comando passando o arquivo que você quer testar:

bash

claude "leia .claudecode/security-agent.md e revise apenas o arquivo"

Use o código com cuidado.



Dica de ouro para o teste: Se quiser ver se ele está funcionando de verdade, pegue um arquivo de testes isolado, escreva de propósito uma query vulnerável concatenando strings no Postgres (ex: `client.query("SELECT * FROM users WHERE email = '" + req.body.email + "'")`) e mande o subagente revisar. Você verá ele entrar, reescrever para a sintaxe parametrizada segura, salvar o arquivo e encerrar a sessão sem gastar um token a mais de histórico [Claude Code].



